MambaCSP: Hybrid-Attention State Space Models for Hardware-Efficient Channel State Prediction

Aladin Djuhera*, Haris Gacanin†, Holger Boche*
*Technical University Munich, Germany, †RWTH Aachen University, Germany
Emails: {aladin.djuhera, boche}@tum.de, harisg@dsp.rwth-aachen.de

***Abstract*—Recent works have demonstrated that attention-based transformer and large language model (LLM) architectures can achieve strong channel state prediction (CSP) performance by capturing long-range temporal dependencies across channel state information (CSI) sequences. However, these models suffer from quadratic scaling in sequence length, leading to substantial computational cost, memory consumption, and inference latency, which limits their applicability in real-time and resource-constrained wireless deployments. In this paper, we investigate whether selective state space models (SSMs) can serve as a hardware-efficient alternative for CSI prediction. We propose MambaCSP, a hybrid-attention SSM architecture that replaces LLM-based prediction backbones with a linear-time Mamba model. To overcome the local-only dependencies of pure SSMs, we introduce lightweight patch-mixer attention layers that periodically inject cross-token attentions, helping with long-context CSI prediction. Extensive MISO-OFDM simulations show that MambaCSP improves prediction accuracy over LLM-based approaches by 9–12%, while delivering up to 3.0x higher throughput, 2.6x lower VRAM usage, and 2.9x faster inference. Our results demonstrate that hybrid state space architectures provide a promising direction for scalable and hardware-efficient AI-native CSI prediction in future wireless networks.**

***Index Terms*—6G, channel state prediction, hardware efficiency, large language models, state space models**

I. Introduction and Motivation

Large language models (LLMs) have gained strong capabilities in coding, math, and complex reasoning, demonstrating their transformative potential across diverse domains. In future 6G networks, they can serve as foundational building blocks for embedding agentic capabilities across communication, connectivity, and artificial intelligence (AI) layers, thereby replacing or augmenting existing workflows [1], [2].

However, not all communication layers can readily benefit from such AI-native integration. Wireless signal processing systems operate under strict real-time constraints, which have traditionally limited the adoption of AI due to high computational and latency requirements [3]. LLMs, in particular, contain billions of parameters and require substantial compute, making them unsuitable for real-time deployment on resource-constrained devices and in latency-sensitive applications where time to first token (TTFT) is critical. Overcoming these constraints is essential for enabling practical AI-native integration.

This work acknowledges support from the German BMFTR through the 6G-life (16KISK002), QD-CamNetz (16KISQ077), QuaPhySI (16KIS1598K), QUIET (16KISQ093), and AISAC (16KIS2462) projects, as well as from the German DFG through CeTI: Centre for Tactile Internet with Human-in-the-Loop (Germany's Cluster of Excellence – EXC 2050/2, ID 390696704). H. Gacanin was supported by the 6GEM research hub (16KISK036K).

Among such latency-sensitive applications is channel state prediction (CSP), which forecasts future channel state information (CSI) from historical observations, thereby reducing overall pilot overhead. However, acquiring CSI via conventional channel estimation methods is challenging, particularly in high-mobility scenarios where shortened channel coherence times significantly increase the estimation overhead. Recent works [4]–[6] have explored LLM-based architectures for CSP, leveraging attention mechanisms to effectively capture long-range, non-local dependencies across time and frequency. Despite their strong performance, transformer-based models exhibit quadratic scaling in sequence length, resulting in substantial computational and memory overhead, as well as increased inference latency due to key-value (KV) caching. This makes them impractical for long-context CSI prediction and real-time deployment in wireless systems.

Selective state space models (SSMs), such as Mamba [7], offer a hardware-efficient alternative to LLMs by modeling sequences without KV caching, resulting in linear complexity. In particular, state transitions are input-dependent and include a continuous-time inductive bias, making SSMs better suited for modeling wireless channel variations. However, pure SSMs process tokens recursively, which can limit their ability to capture non-local dependencies in complex CSI sequences.

In this paper, we address these limitations and propose **MambaCSP**, a hybrid-attention state space architecture for hardware-efficient CSP. Our approach employs a Mamba-based backbone with lightweight patch-mixer attention layers that periodically inject cross-token attention, enabling modeling of long-range dependencies across CSI sequences while preserving the efficiency of SSMs. To this end, we adapt and generalize LLM-based CSP pipelines into a unified framework that enables efficient processing of CSI sequences across both model architectures. Our key findings are:

- **Prediction Performance**: MambaCSP consistently outperforms LLM-based and other neural baselines across diverse mobility scenarios in both TDD and FDD settings.
- **Hardware Efficiency**: MambaCSP achieves up to 3.0× higher throughput, 2.6× lower memory usage, and 2.9× faster inference compared to LLM-based prediction.
- **Hybrid-Attention Design**: MambaCSP's patch-mixer attention is particularly beneficial in FDD scenarios, where non-local, cross-frequency dependencies require explicit token interactions beyond recursive state-space modeling.

This paper is organized as follows. Section II presents the system model and formalizes the problem statement. Section III introduces the CSI prediction pipeline and MambaCSP. Section IV describes the experiment setup. Section V discusses the simulation results, and Section VI concludes the paper.

II. System Model and CSI Prediction Task

In this section, we formally define the wireless system model and the corresponding channel state prediction task.

A. Wireless System Model

We consider a single-cell MISO-OFDM link, where a base station (BS) equipped with a uniform planar array (UPA) with N_t antennas serves U single-antenna user equipments (UEs). We follow a 5G NR FR1 OFDM numerology with carrier frequency $f_c = 2.4$ GHz, subcarrier spacing $\Delta f = 15$ kHz, and $N_{\rm sc} = 12$ subcarriers per resource block (RB), such that each RB has a bandwidth of $B_{\rm RB} = N_{\rm sc}\Delta f = 180$ kHz. We consider two contiguous frequency bands of equal bandwidth for uplink (UL) and downlink (DL), each consisting of $K_{\rm UL} = K_{\rm DL} = 48$ RBs, resulting in a per-band bandwidth of 8.64 MHz. Time is organized in slots of duration $T_{\rm slot} = 1$ ms and users move with velocities in the range of $v \in [10, 100]$ km/h.

For each UE, we consider a sequence of $N_{\rm slot} = P + L$ consecutive slots, where the first P slots form a *history window* and the remaining L slots form a *prediction window*. For the UL band, let $k \in \{1, \ldots, K_{\rm UL}\}$ denote the RB index and $s \in \{1, \ldots, N_{\rm slot}\}$ the slot index. The corresponding narrowband MISO channel vector on RB k and slot s is denoted as

$$\mathbf{h}^{\rm UL}_{k,s} \in \mathbb{C}^{N_t} \ . \tag{1}$$

Similarly, for the DL band, we define $\mathbf{h}^{\rm DL}_{k,s} \in \mathbb{C}^{N_t}$.

We model the wireless channel as frequency-selective and time-varying, incorporating multipath propagation and Doppler effects induced by user mobility. The corresponding parametric representation of the UL channel is thus given by

$$\mathbf{h}^{\rm UL}_{k,s} = \sum_{\ell=1}^{L_p} \alpha_\ell \mathbf{a}_{\rm BS}(\theta_\ell, \phi_\ell) e^{-j2\pi f_k \tau_\ell} e^{j2\pi f_{D,\ell} t_s} \ , \tag{2}$$

where α_ℓ, τ_ℓ, and (θ_ℓ, ϕ_ℓ) denote the complex gain, delay, and angles of departure of path ℓ, $\mathbf{a}_{\rm BS}(\cdot)$ is the BS array response, and $f_{D,\ell}$ is the Doppler shift. We consider both time-division duplex (TDD) and frequency-division duplex (FDD) modes.

B. CSI-RS and DMRS Configuration

Channel estimation in 5G NR follows the *Demodulation Reference Signal* (DMRS) pilot pattern, where pilots are transmitted on a sparse subset of RBs and symbols. Formally, let $\mathcal{K}_{\rm CSI} \subseteq \{1, \ldots, K_{\rm UL}\}$ denote the set of RBs carrying pilot signals and let $\mathcal{S}_{\rm CSI} \subseteq \{1, \ldots, P\}$ denote the corresponding time indices. The observed CSI is given by

$$\tilde{\mathbf{h}}^{\rm UL}_{k,s} = \mathbf{h}^{\rm UL}_{k,s} + \mathbf{n}_{k,s} \ , \quad (k,s) \in \mathcal{K}_{\rm CSI} \times \mathcal{S}_{\rm CSI} \ , \tag{3}$$

where $\mathbf{n}_{k,s} \sim \mathcal{CN}(0, \sigma_n^2 \mathbf{I})$ models estimation noise. Note that different DMRS pilot patterns can be configured depending on the scenario, controlling the corresponding pilot density.

C. Channel State Prediction Task

Given the UL history window over P slots, we aim to predict the future DL CSI over the next L slots across all RBs. To this end, let the UL history be defined as

$$\mathcal{H}^{\rm UL}_{\rm his} = \{\tilde{\mathbf{h}}^{\rm UL}_{k,s} \mid k \in \mathcal{K}_{\rm CSI}, s \in \mathcal{S}_{\rm CSI}\} \tag{4}$$

with corresponding DL prediction target

$$\mathcal{H}^{\rm DL}_{\rm pre} = \{\mathbf{h}^{\rm DL}_{k,s} \mid k \in \{1, \ldots, K_{\rm DL}\}, s \in \{P+1, \ldots, P+L\}\} \ , \tag{5}$$

This defines a sequence-to-sequence prediction of the form

$$\hat{\mathcal{H}}^{\rm DL}_{\rm pre} = f_\Theta(\mathcal{H}^{\rm UL}_{\rm his}) \ , \tag{6}$$

where f_Θ denotes a parameterized model with learnable parameters Θ, whose prediction performance is evaluated using the normalized mean square error (NMSE), i.e.,

$$\text{NMSE} = \mathbb{E}\left[\frac{\sum_{k,s} \|\hat{\mathbf{h}}^{\rm DL}_{k,s} - \mathbf{h}^{\rm DL}_{k,s}\|_2^2}{\sum_{k,s} \|\mathbf{h}^{\rm DL}_{k,s}\|_2^2}\right]. \tag{7}$$

In our work, we require f_Θ to not only accurately predict future DL CSI, but also to minimize the computational complexity and latency, enabling real-time deployment.

III. MambaCSP: Hybrid-Attention State Space Models for CSI Prediction

In this section, we introduce a unified framework for CSI prediction that is compatible with both LLM and SSM sequence models. We then present the proposed hybrid MambaCSP architecture, which combines the efficiency of SSMs with the expressivity of attention mechanisms.

A. Unified CSI Prediction Framework

Building on prior works [4], [5], we generalize attention-based CSP pipelines into a unified framework that supports arbitrary sequence modeling architectures, including SSMs (see Fig. 1). The key components are outlined as follows.

1) Data Preprocessing: CSI data is complex-valued and needs to be converted into real-valued embeddings first. To this end, we extract frequency- and delay-domain representations, normalize the data, and partition it into temporal patches. We denote the historical UL CSI over P slots and K RBs as $\mathbf{H}_f \in \mathbb{C}^{K \times P}$ and its delay-domain representation as $\mathbf{H}_\tau = \mathbf{F}_K^H \mathbf{H}_f$, where $\mathbf{F}_K$ denotes the K-point DFT matrix. We then separate real and imaginary components and obtain $\mathbf{X}_f, \mathbf{X}_\tau \in \mathbb{R}^{2 \times K \times P}$. To ensure stable training across varying SNR conditions, we normalize each representation and reshape the tensors by merging feature dimensions, yielding $\tilde{\mathbf{X}}_f, \tilde{\mathbf{X}}_\tau \in \mathbb{R}^{2K \times P}$. To further reduce sequence length and computational complexity, we partition the temporal dimension into non-overlapping patches of size N (see Fig. 1a), resulting in

$$\mathbf{X}^p_f, \mathbf{X}^p_\tau \in \mathbb{R}^{2K \times N \times P'}, \tag{8}$$

where $P' = \lceil P/N \rceil$ denotes the number of patches.

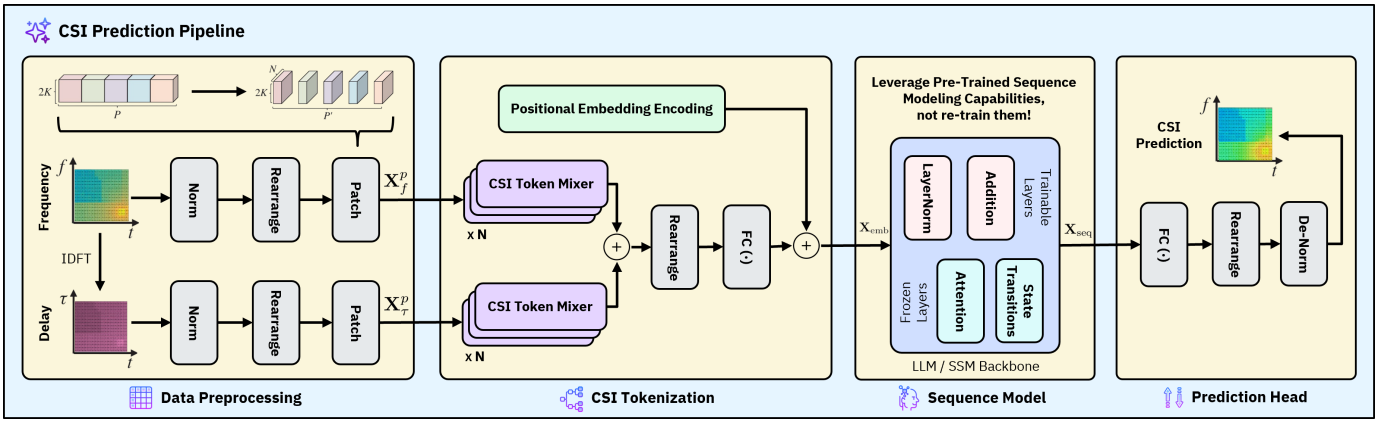

Fig. 1: CSI prediction pipeline: Historical UL CSI is converted into frequency/delay components, normalized, rearranged, and partitioned into temporal patches. CSI token mixer blocks then produce token embeddings for the sequence model. A final prediction head maps the outputs back to the complex CSI domain.

2) CSI Tokenization: Preprocessed CSI patches need to be further tokenized into embeddings that are compatible with the input format of language models. To this end, each patch is processed by a *CSI Token Mixer* (see Fig. 2), which adaptively reweights features to emphasize informative multipath components while attenuating less relevant or noisy ones.

Let $\mathbf{X}_i \in \mathbb{R}^{2K\times N\times P'}$ denote the input tensor of a token mixer block. First, local feature extraction via convolutional $\mathrm{Conv}(\cdot)$ and $\mathrm{ReLU}(\cdot)$ layers is performed to capture joint temporal and frequency-domain dependencies, i.e.,

$$\mathbf{X}_{\text{feat}} = \mathrm{Conv}(\mathrm{ReLU}(\mathrm{Conv}(\mathbf{X}_i))) \ . \tag{9}$$

To model the relative importance of each token, a channel-wise gating mechanism with sigmoid activation $\sigma(\cdot)$ is applied

$$\mathbf{X}_{\text{gate}} = \sigma(\mathrm{FC}(\mathrm{ReLU}(\mathrm{FC}(\mathrm{GAP}(\mathbf{X}_{\text{feat}}))))) \ , \tag{10}$$

where $\mathrm{GAP}(\cdot)$ and $\mathrm{FC}(\cdot)$ denote global average pooling and fully connected layers. The resulting weights $\mathbf{X}_{\text{gate}} \in \mathbb{R}^{1\times 1\times P'}$ are then used to reweight each token, i.e.,

$$\mathbf{X}_{\text{sca}}[:,:,i] = \mathbf{X}_{\text{gate}}[i] \cdot \mathbf{X}_{\text{feat}}[:,:,i] \ , \tag{11}$$

before applying a residual connection, which preserves the original features and stabilizes gradient flow during training

$$\mathbf{X}_o = \mathbf{X}_{\text{sca}} + \mathbf{X}_i \ . \tag{12}$$

Next, to jointly capture spectral correlations and multipath structure, we cascade multiple CSI Token Mixer blocks over the frequency/delay representations, and combine them as

$$\mathbf{X}_{\text{tok}} = \mathrm{TM}^{(N)}(\mathbf{X}_f^p) + \mathrm{TM}^{(N)}(\mathbf{X}_\tau^p) \ , \tag{13}$$

where $\mathrm{TM}^{(N)}(\cdot)$ denotes N cascaded token mixer blocks. In practice, we employ $N \in [2,4]$ to balance modeling capacity and computational efficiency. The resulting tensor is rearranged into $\tilde{\mathbf{X}}_{\text{tok}} \in \mathbb{R}^{2KN\times P'}$ and projected into the model embedding dimension, yielding $\bar{\mathbf{X}}_{\text{tok}} \in \mathbb{R}^{F\times P'}$.

Furthermore, to preserve the temporal ordering, we follow [4] and add sinusoidal positional encodings defined as

$$\mathbf{X}_{\text{PE}}(i,j) = \begin{cases} \sin\left(\frac{j}{10000^{i/F}}\right) \ , & i \text{ even} \ , \\ \cos\left(\frac{j}{10000^{(i-1)/F}}\right) \ , & i \text{ odd} \ , \end{cases} \tag{14}$$

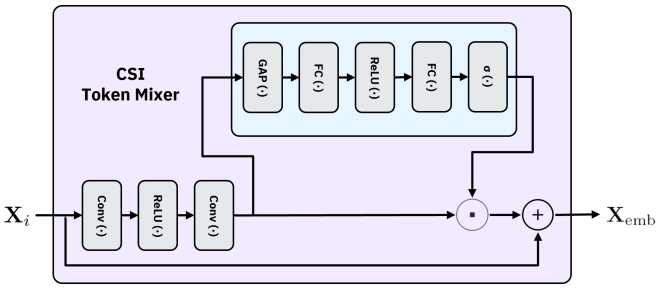

Fig. 2: CSI token mixer module.

where i indexes the feature dimension and j the token position. The final CSI token embeddings are then obtained as

$$\mathbf{X}_{\text{emb}} = \bar{\mathbf{X}}_{\text{tok}} + \mathbf{X}_{\text{PE}} \ . \tag{15}$$

3) Sequence Model Backbone: To model spatio-temporal dependencies across CSI token sequences, we leverage the pre-trained backbone's capacity to attend to complex multi-scale interactions rather than its learned semantic world knowledge. During training, we therefore keep the backbone's core layers frozen while only adapting lightweight task-specific components. In LLMs, for example, this corresponds to freezing the multi-head attention and feed-forward blocks, while only fine-tuning LayerNorm and Addition layers [4], [5]. This significantly reduces the overall training overhead and yields contextualized token representations given by

$$\mathbf{X}_{\text{seq}} = f_{\Theta_{\text{bb}}}(\mathbf{X}_{\text{emb}}) \in \mathbb{R}^{F\times P'} \ , \tag{16}$$

where $f_{\Theta_{\text{bb}}}(\cdot)$ denotes the sequence model backbone, and $\mathbf{X}_{\text{seq}}$ captures the learned spatio-temporal dependencies.

4) Prediction Head: In the final step, $\mathbf{X}_{\text{seq}}$ is mapped back to its original domain. We first apply a linear projection to obtain $\hat{\mathbf{X}} \in \mathbb{R}^{2K\times L}$ and reshape the predicted tensor into real/imaginary components of the form $\hat{\mathbf{X}} \in \mathbb{R}^{2\times K\times L}$. We then de-normalize to recover the original CSI scale, i.e.,

$$\hat{\mathbf{X}}_{\text{de}} = \sigma\,\hat{\mathbf{X}} + \mu \ , \tag{17}$$

where μ and σ denote the mean and standard deviation used during preprocessing. Finally, the predicted CSI is obtained as

$$\hat{\mathbf{H}}^{\text{DL}} = \hat{\mathbf{X}}_{\text{de}}[1,:,:] + j\,\hat{\mathbf{X}}_{\text{de}}[2,:,:] \ . \tag{18}$$

B. Hybrid Mamba Architecture

Mamba [7] extends classical SSMs by introducing input-dependent state transitions. Instead of fixed matrices $\mathbf{A}$ (state transition), $\mathbf{B}$ (input projection), and $\mathbf{C}$ (output projection), Mamba parameterizes these operators as functions of the input:

$$\mathbf{h}_t = \mathbf{A}(\mathbf{x}_t)\mathbf{h}_{t-1} + \mathbf{B}(\mathbf{x}_t)\mathbf{x}_t \ , \quad \mathbf{y}_t = \mathbf{C}(\mathbf{x}_t)\mathbf{h}_t \ . \tag{19}$$

This allows Mamba to dynamically control how information is propagated and updated at each time step, improving its ability to model non-stationary sequences. In addition, Mamba incorporates a continuous-time inductive bias via learned step sizes Δt, enabling adaptive temporal scaling of the state updates. This is particularly well-suited for Doppler-induced channel variations, which evolve continuously over time.

However, Mamba propagates information recursively through the hidden state. As a result, information from distant tokens must be compressed into $\mathbf{h}_t$, which can lead to degradation over long horizons. Consequently, Mamba does not explicitly model pairwise interactions across the entire sequence as attention-based models do. This can be limiting for CSI prediction, particularly in *a) high-mobility scenarios*, where rapid channel variations introduce non-local dependencies across time and frequency, and in *b) FDD settings*, where predicting DL CSI from UL CSI requires modeling complex cross-frequency relationships spanning distant tokens.

To address these limitations, we propose **MambaCSP**, a *hybrid-attention* extension of the Mamba backbone (see Fig. 3). Specifically, we introduce sparse *patch-mixer attention layers*, which operate on the tokenized CSI patches every k blocks and perform lightweight cross-token interactions using a small number of attention heads H (typically 2–4). Given $\mathbf{X}_{\text{emb}} \in \mathbb{R}^{F \times P'}$, patch-mixer attention is computed as

$$\text{Attn}(\mathbf{X}_{\text{emb}}) = \text{softmax}\left(\frac{\mathbf{Q}^\top \mathbf{K}}{\sqrt{d_h}}\right)\mathbf{V}^\top \ , \tag{20}$$

where d_h denotes the head dimension and $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ are the query, key, and value projections obtained from $\mathbf{X}_{\text{emb}}$ via learned linear mappings. The resulting representations are concatenated and projected back via $\mathbf{W}_O \in \mathbb{R}^{F \times F}$, i.e.,

$$\mathbf{X}_{\text{mix}} = \text{Concat}\big(\text{Attn}_1, \ldots, \text{Attn}_H\big)\mathbf{W}_O \ . \tag{21}$$

The attention output is then inserted sparsely every k Mamba blocks through a residual update, i.e.,

$$\mathbf{X}^{(\ell+1)} = \mathbf{X}^{(\ell)} + \mathbf{X}^{(\ell)}_{\text{mix}} \ , \qquad \ell \in \{k, 2k, \ldots\} \ . \tag{22}$$

This restores global context modeling while maintaining near-linear complexity. During training, patch-mixer attentions are learned jointly with projection and LayerNorm layers, while the majority of the Mamba backbone remains frozen.

C. Computational Complexity and Efficiency

Plain Mamba's state-space updates process sequences one step at a time with recurrence. Thus, for a token sequence of length P' and hidden dimension F, compute scales linearly as $\mathcal{O}(P'F)$. Moreover, since Mamba does not construct a full pairwise attention matrix and does not require KV caching, its memory usage also scales linearly with sequence length.

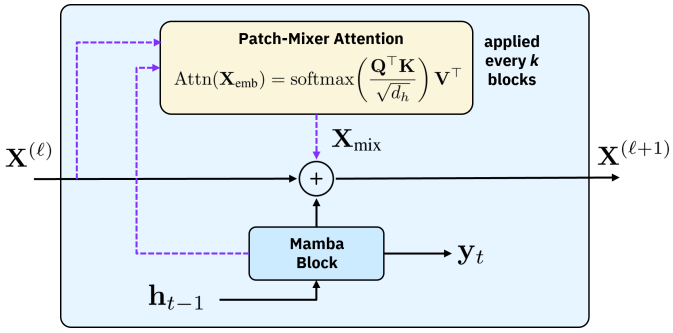

Fig. 3: Hybrid-attention MambaCSP architecture at layer l.

For our hybrid MambaCSP, patch-mixer attentions are inserted only intermittently and use a small number of heads. Thus, for L_M Mamba blocks and $\lfloor L_M/k \rfloor$ patch-mixer layers, where in practice $\lfloor L_M/k \rfloor \ll L_M$, the overall complexity is

$$\mathcal{O}(L_M P' F) + \mathcal{O}\left(\left\lfloor \tfrac{L_M}{k} \right\rfloor P'^2 F\right) \ \approx \ \mathcal{O}(L_M P' F) \ , \tag{23}$$

such that MambaCSP retains near-linear scaling overall.

In contrast, LLM backbones require self-attention in *every layer*. As a result, the forward computational cost scales quadratically as $\mathcal{O}(P'^2 F)$, while memory is dominated by the attention matrix of size $P' \times P'$, i.e., $\mathcal{O}(P'^2)$. Although freezing transformer layers reduces the number of trainable parameters during training, it does not remove the quadratic self-attention computation in the forward pass. Therefore, MambaCSP provides a more favorable complexity–efficiency trade-off for long CSI histories and low-latency deployment.

IV. Experimental Setup

To evaluate MambaCSP, we generate a comprehensive CSI dataset with QuaDRiGa [8] for the 3GPP TR 38.901 UMa NLOS channel. The OFDM numerology and all remaining parameters follow our system model in Sec. II. The BS is placed at $(0, 0, 30)$ m and is equipped with a 4×4 dual-polarized UPA. For each UE, we simulate a frame of $N_{\text{slot}} = 20$ slots, where the first $P = 16$ slots form the history window and the remaining $L = 4$ slots form the prediction window. For each speed realization, we randomly place 10 UEs in an annulus between 20 m and 50 m around a cluster center at $(200, 0, 1.5)$ m and generate linear user tracks. For DMRS, we employ a frequency-selective pilot pattern with temporal spacing $\Delta t_{\text{DMRS}} = 2\Delta t$ and frequency spacing $\Delta k = 1$ RB. In total, we generate 8000 training, 2000 validation, and 10000 test samples for both TDD and FDD[1].

We benchmark our hybrid MambaCSP against plain Mamba, LLM-based prediction [4], and classical neural baselines including CNN [9], RNN [10], and LSTM [11]. To ensure fairness, we employ a 130M parameter size Mamba 2 model [12], whose scale is comparable to the 125M parameter size GPT-2 [13] model used in [4]. We further note that increasing the backbone model size or changing the architecture,

[1]Training code and dataset available at: XXX

e.g., to GPT-OSS [14] or other larger models, yields at most marginal gains for CSP, since the task does not primarily rely on semantic world knowledge or reasoning abilities (see Table III). We train each model for 10 epochs using Adam with $\beta = (0.9, 0.999)$, batch size 256, and learning rate 10^{-3}.

V. Results and Discussions

We evaluate MambaCSP in terms of prediction accuracy and hardware efficiency, and perform additional ablations.

A. Prediction Accuracy

Fig. 4 and Fig. 5 compare the NMSE of all considered baselines for the TDD and FDD settings, respectively, showing that MambaCSP outperforms LLM-based CSP in both duplex modes. For TDD, MambaCSP achieves 5% lower NMSE than plain Mamba and 9% lower NMSE than the LLM, indicating that state-space modeling already captures most relevant temporal structures in TDD, while patch-mixer attention provides a moderate but non-negligible gain. For FDD, the relative benefit of MambaCSP becomes more pronounced. It consistently outperforms plain Mamba by 17% and the LLM by 12% across the full velocity range, showing that patch-mixer attention is particularly beneficial in FDD when non-local cross-frequency dependencies arise due to the UL-to-DL prediction gap, which causes plain Mamba to underperform, particularly at higher velocities. Furthermore, MambaCSP, plain Mamba, and the LLM significantly outperform CNN, RNN, and LSTM baselines, suggesting that sequence models are better at predicting spatio-temporal CSI dependencies.

B. Throughput

Fig. 6 shows the training throughput as the sequence length P' increases. As expected, throughput decreases noticeably as the input history becomes longer. Overall, LLM-based CSP degrades much faster than Mamba-based variants. Specifically, MambaCSP achieves $2.7\times$, $2.9\times$, and $3.0\times$ higher throughput than the LLM at $P' \in \{32, 48, 64\}$, respectively, which is similar to plain Mamba. This shows that the proposed patch-level attention introduces only a minor throughput overhead.

C. Memory Consumption

Fig. 7 reports the peak memory usage when increasing the sequence length P'. In line with our theoretical observations, the LLM exhibits a much steeper increase in memory consumption, driven by the quadratic attention matrix and KV caching. For short input histories up to $P' = 8$, the memory gap between LLM and Mamba-based models remains approximately constant, but it widens steadily as P' grows. Specifically, the LLM requires $2.0\times$, $2.3\times$, and $2.6\times$ more VRAM than the Mamba variants for $P' \in \{32, 48, 64\}$. In contrast, plain Mamba and MambaCSP remain close and scale linearly in memory, substantiating our analysis that patch-level attention incurs only a negligible memory overhead.

D. Inference Latency

Fig. 8 compares the inference latency per forward pass across sequence lengths. Similar to memory usage, latency increases with P' for all models. MambaCSP is about $2.0\times$,

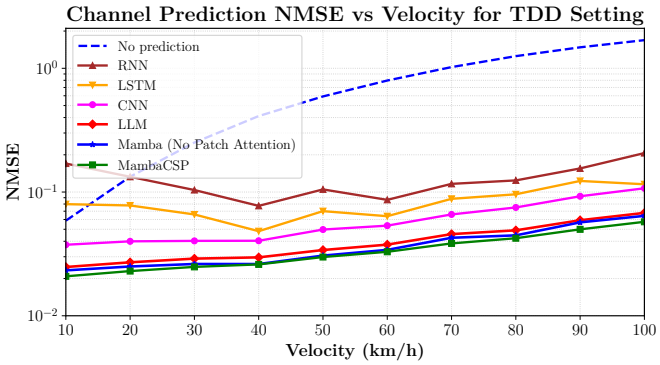

Fig. 4: NMSE results for TDD. Across all approaches, the NMSE increases with increasing velocity. MambaCSP outperforms plain Mamba and the LLM.

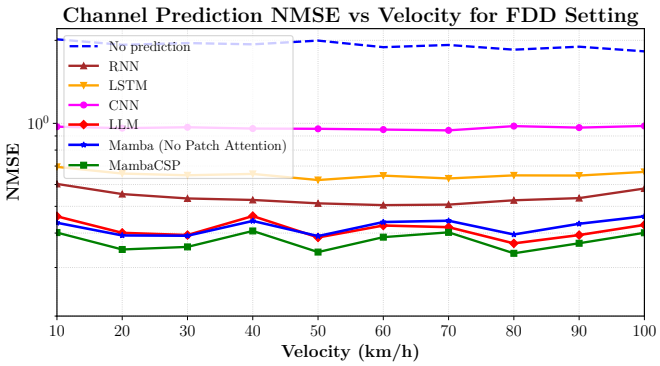

Fig. 5: NMSE results for FDD. Across all approaches, the NMSE remains approximately flat. MambaCSP outperforms plain Mamba and the LLM.

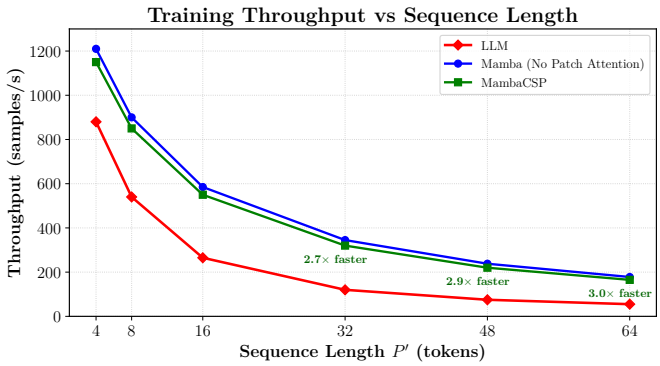

Fig. 6: Throughput for different sequence lengths. MambaCSP achieves up to $3\times$ more samples/s for longer CSI patches, comparable to plain Mamba.

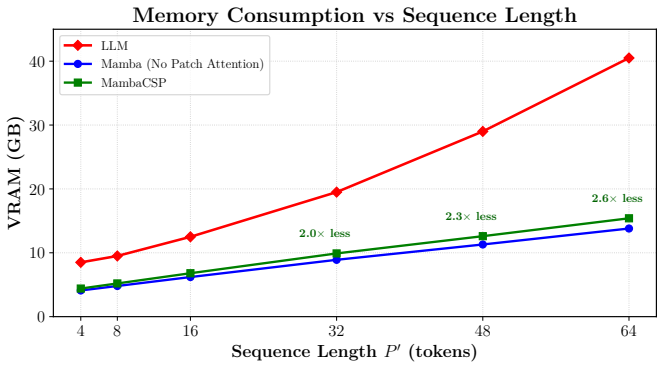

Fig. 7: Memory consumption for different sequence lengths. LLM grows faster with sequence length due to KV caching. Mamba backbones remain linear.

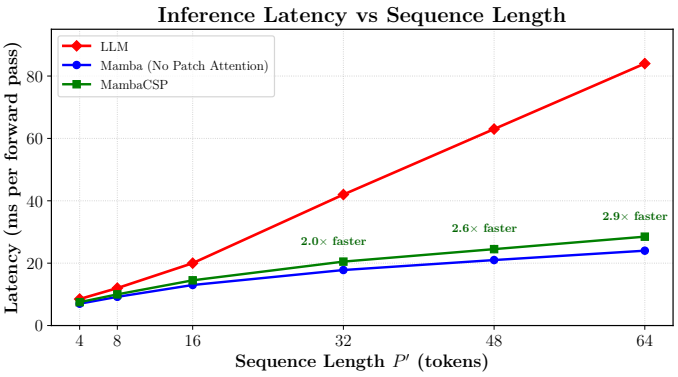

Fig. 8: Latency for different sequence lengths. Mamba backbones scale linearly with longer sequence lengths compared to LLM (up to 3× slower).

2.6×, and 2.9× faster than the LLM at $P' \in \{32, 48, 64\}$, respectively, while remaining close to plain Mamba, thereby preserving the latency benefits of the state-space backbone.

E. NMSE vs Input Sequence Length

Table I reports the FDD NMSE for MambaCSP and LLM backbones as the input sequence length P' increases. Overall, a longer CSI history improves prediction (with diminishing returns at $P' = 32$) as additional past observations expose more spatio-temporal structure that is useful for inferring future DL CSI. This supports our motivation for increasing P', especially in FDD, which benefits from longer-range CSI dependencies. Similar trends can be observed for TDD.

F. Ablations on MambaCSP Patch-Mixer Attention Layers

Table II ablates the impact of the patch-mixer attention layers in MambaCSP. Overall, injecting cross-attention with $H = 2$ heads every $k = 4$ Mamba blocks is sufficient to restore non-local sequence modeling capabilities for complex CSI patches, while providing a favorable trade-off between prediction accuracy and efficiency. Further increasing the number of heads or inserting attention more frequently yields only marginal additional gains at the cost of increased complexity.

In summary, our results demonstrate that MambaCSP achieves a substantially better accuracy–efficiency trade-off than LLM-based CSP, particularly for longer CSI histories. Its lower latency and memory footprint make it especially attractive for real-time deployment on low-resource devices. Further reducing memory and compute through quantization and model pruning is an interesting direction for future work.

VI. Conclusion

In this paper, we proposed MambaCSP, a hybrid-attention state space model architecture as a hardware-efficient alternative to LLM-based CSI prediction. MambaCSP replaces the LLM backbone in existing CSP pipelines with a Mamba-based sequence model and augments it with sparse patch-mixer attention layers to better capture long-range CSI token dependencies while preserving the linear-time efficiency of state space models. Our experiments show that MambaCSP achieves better accuracy than LLM-based prediction in both TDD and FDD settings, while providing substantially higher throughput, lower memory consumption, and lower inference latency. Our findings thus point to a key takeaway: LLMs are not the final architectural choice for long-context CSI prediction, and combining efficient state space models with lightweight attention is a promising path toward scalable and real-time AI-native CSI prediction in future wireless systems.

TABLE I: NMSE versus sequence length P' for LLM and MambaCSP (FDD). Increasing P' improves prediction with diminishing returns at larger contexts.

Seq. Length P'	LLM NMSE	MambaCSP NMSE	Relative Gain
4	0.503	0.468	7.0%
8	0.462	0.422	8.7%
16	0.425	0.366	13.9%
32	0.392	0.331	15.6%
48	0.381	0.318	16.5%
64	0.374	0.309	17.4%

TABLE II: Ablation of the patch-mixer attention layers in MambaCSP. Sparse attention improves FDD more than TDD. In practice, we use $k = 4$ and $H = 2$ as a favorable accuracy-efficiency trade-off.

Patch-Mixer	Interval k	Heads H	TDD NMSE ↓	FDD NMSE ↓
None	–	–	0.0374	0.4222
Yes	4	2	0.0346	0.3750
Yes	4	4	**0.0345**	**0.3736**
Yes	2	4	0.0345	0.3731

TABLE III: Comparison of different backbones. Larger models provide only marginal gains for CSP despite substantially higher memory requirements.

Backbone	Params	VRAM (GB)	TDD NMSE ↓	FDD NMSE ↓
GPT-2 Small	124M	6.8	0.050	0.425
GPT-OSS	1.3B	18.9	0.049	0.423
Mamba2-130M	130M	5.1	0.049	0.422
Mamba2-780M	780M	11.8	0.048	0.420

References

[1] F. Jiang, C. Pan, K. Wang, P. Michiardi, O. A. Dobre, and M. Debbah, "From Large AI Models to Agentic AI: A Tutorial on Future Intelligent Communications," *IEEE JSAC*, 2026.

[2] A. Djuhera, F. Koch, and A. Binotto, "Joint Partitioning and Placement of Foundation Models for Real-Time Edge AI," *IEEE ICNC*, 2026.

[3] J. Guo, Y. Cui, S. Jin, and J. Zhang, "Large AI Models for Wireless Physical Layer," *IEEE Communications Magazine*, 2026.

[4] B. Liu, X. Liu, S. Gao, X. Cheng, and L. Yang, "LLM4CP: Adapting Large Language Models for Channel Prediction," *Journal of Communications and Information Networks*, vol. 9, no. 2, pp. 113–125, 2024.

[5] S. Fan, Z. Liu, X. Gu, and H. Li, "CSI-LLM: A novel Downlink Channel Prediction Method Aligned with LLM Pre-Training," in *IEEE Wireless Communications and Networking Conference*, 2025.

[6] Y. Cui, J. Guo, C.-K. Wen, S. Jin, and E. Tong, "Exploring the Potential of Large Language Models for Massive MIMO CSI Feedback," *arXiv preprint arXiv:2501.10630*, 2025.

[7] A. Gu and T. Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," in *COLM*, 2024.

[8] S. Jaeckel, L. Raschkowski, K. Börner, and L. Thiele, "QuaDRiGa: A 3-D Multi-Cell Channel Model with Time Evolution for Enabling Virtual Field Trials," *IEEE Transactions on Antennas and Propagation*, 2014.

[9] M. S. Safari, V. Pourahmadi, and S. Sodagari, "Deep UL2DL: Data-Driven Channel Knowledge Transfer from Uplink to Downlink," *IEEE Open Journal of Vehicular Technology*, 2019.

[10] W. Jiang and H. D. Schotten, "Neural Network-Based Fading Channel Prediction: A Comprehensive Overview," *IEEE Access*, 2019.

[11] ——, "Deep Learning for Fading Channel Prediction," *IEEE Open Journal of the Communications Society*, 2020.

[12] T. Dao and A. Gu, "Mamba-2 130M Model Weights," https://huggingface.co/state-spaces/mamba2-130m, 2024.

[13] A. Radford *et al.*, "Language Models are Unsupervised Multitask Learners," *OpenAI Blog*, 2019.

[14] OpenAI, "GPT-OSS-120B & GPT-OSS-20B Model Card," 2025. [Online]. Available: https://arxiv.org/abs/2508.10925